



Assignment

Course:

Recursion Assignment

Ronit Mexson

Registration Number: 24BAI0036

Theory Slot : E2 + TE2

Lab Slot : L15 + L16

Professor: Dr. Jagalingam P



Question No. 1

Problem Statement

Two players, Player 1 (Maximizer) and Player 2 (Minimizer), play a game on an $M \times N$ matrix of integers. The board contains both positive and negative scores.

Game Rules:

Player 1 starts at cell (0, 0), and Player 2 starts at (0, N-1). The starting cells are considered visited, and their values are part of the players' initial state.

Player 1 moves first. The players take turns moving their token to an unvisited adjacent cell (up, down, left, or right).

The game ends when the current player cannot make a valid move (i.e., all adjacent cells are either visited or out-of-bounds).

Player 1's score is the sum of values of all cells they have visited. Player 2 does not have their own score; their sole purpose is to make moves that minimize Player 1's final score. Player 1's goal is to maximize their final score.

Assuming both players play optimally, what is Player 1's final score?

Input Format

A 2D integer matrix (or list of lists) representing the game board.

Output Format

A single integer representing Player 1's final score, assuming optimal play from both sides.

Code

```
#include<stdio.h>
#define max 100

int m,n;
int board[max][max];
int visited[max][max];

int dp[max][max][max][max][2];
int calculated[max][max][max][max][2];

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

int solve(int x1, int y1, int x2, int y2, int turn, int score1);

int myMax(int a, int b){
    if(a > b){
```

```

        return a;
    }
    else{
        return b;
    }
}

int myMin(int a, int b){
    if(a < b){
        return a;
    }
    else{
        return b;
    }
}

void readMatrix(int i, int j){
    if(i == m){
        return;
    }
    if(j == n){
        readMatrix(i + 1, 0);
        return;
    }
    scanf("%d",&board[i][j]);
    visited[i][j] = 0;
    readMatrix(i,j+1);
}

int exploreMoves(int d, int x1, int y1, int x2, int y2, int turn, int score1, int best, int *moved){
    if(d == 4){
        return best;
    }

    int nx, ny;

    if(turn == 0){
        nx = x1 + dx[d];
        ny = y1 + dy[d];
    }else{
        nx = x2 + dx[d];
        ny = y2 + dy[d];
    }

    if(nx >= 0 && nx < m && ny >= 0 && ny < n && visited[nx][ny] == 0){
        *moved = 1;
        visited[nx][ny] = 1;

        int val;
        if(turn == 0){
            val = solve(nx,ny,x2,y2,1,score1 + board[nx][ny]);
            best = myMax(best,val);
        }
        else{
            val = solve(x1,y1,nx,ny,0,score1);
            best = myMin(best,val);
        }
    }
}

```

```

        visited[nx][ny] = 0;
    }

    return exploreMoves(d + 1, x1, y1, x2, y2, turn, score1, best, moved);
}

int solve(int x1, int y1, int x2, int y2, int turn, int score1){
    if(calculated[x1][y1][x2][y2][turn] == 1){
        return dp[x1][y1][x2][y2][turn];
    }

    int best;
    int moved = 0;

    if(turn == 0){
        best = -10000000;
        best = exploreMoves(0,x1,y1,x2,y2,turn,score1,best,&moved);
        if(moved == 0){
            best = score1;
        }
    }
    else{
        best = 10000000;
        best = exploreMoves(0,x1,y1,x2,y2,turn,score1,best,&moved);
        if(moved == 0){
            best = score1;
        }
    }
    calculated[x1][y1][x2][y2][turn] = 1;
    dp[x1][y1][x2][y2][turn] = best;
    return best;
}

int main(){
    scanf("%d %d", &m,&n);
    readMatrix(0,0);
    visited[0][0] = 1;
    visited[0][n-1] = 1;

    int result = solve(0,0,m-1,n-1,0,board[0][0]);

    printf("%d\n",result);
    return 0;
}

```

Screenshot Code

```
1.c 1
1  #include<stdio.h>
2  #define max 100
3
4  int m,n;
5  int board[max][max];
6  int visited[max][max];
7
8  int dp[max][max][max][max][2];
9  int calculated[max][max][max][max][2];
10
11 int dx[4] = {-1, 1, 0, 0};
12 int dy[4] = {0, 0, -1, 1};
13
14 int solve(int x1, int y1, int x2, int y2, int turn, int score1);
15
16 int myMax(int a, int b){
17     if(a > b){
18         return a;
19     }
20     else{
21         return b;
22     }
23 }
24
25 int myMin(int a, int b){
26     if(a < b){
27         return a;
28     }
29     else{
30         return b;
31     }
32 }
33
34 void readMatrix(int i, int j){
35     if(i == m){
36         return;
37     }
38     if(j == n){
39         readMatrix(i + 1, 0);
40         return;
41     }
42     scanf("%d",&board[i][j]);
43     visited[i][j] = 0;
44     readMatrix(i,j+1);
45 }
46
47
48 int exploreMoves(int d, int x1, int y1, int x2, int y2, int turn, int score1){
49     if(d == 4){
50         return best;
51     }
52     int nx, ny;
53
54     if(turn == 0){
55         nx = x1 + dx[d];
56         ny = y1 + dy[d];
57     }else{
58         nx = x2 + dx[d];
59         ny = y2 + dy[d];
60     }
61
62     if(nx >= 0 && nx < m && ny >= 0 && ny < n && visited[nx][ny] == 0){
63         *moved = 1;
64         visited[nx][ny] = 1;
65
66         int val;
67         if(turn == 0){
68             val = solve(nx,ny,x2,y2,1,score1 + board[nx][ny]);
69             best = myMax(best,val);
70         }
71         else{
72             val = solve(x1,y1,nx,ny,0,score1);
73             best = myMin(best,val);
74         }
75
76         visited[nx][ny] = 0;
77     }
78
79     return exploreMoves(d + 1, x1, y1, x2, y2, turn, score1, best, moved);
80 }
81
82 int solve(int x1, int y1, int x2, int y2, int turn, int score1){
83     if(calculated[x1][y1][x2][y2][turn] == 1){
84         return dp[x1][y1][x2][y2][turn];
85     }
86
87     int best;
88     int moved = 0;
89
90     if(turn == 0){
91         best = -10000000;
92         best = exploreMoves(0,x1,y1,x2,y2,turn,score1,best,&moved);
93     }
```

```

93     if(moved == 0){
94         best = score1;
95     }
96 }
97 else{
98     best = 10000000;
99     best = exploreMoves(0,x1,y1,x2,y2,turn,score1,best,&moved);
100     if(moved == 0){
101         best = score1;
102     }
103 }
104 calculated[x1][y1][x2][y2][turn] = 1;
105 dp[x1][y1][x2][y2][turn] = best;
106 return best;
107 }
108
109 int main(){
110     scanf("%d %d", &m,&n);
111     readMatrix(0,0);
112     visited[0][0] = 1;
113     visited[0][n-1] = 1;
114
115     int result = solve(0,0,m-1,n-1,0,board[0][0]);
116
117     printf("%d\n",result);
118     return 0;
119 }

```

Ln 16, Col 25 C

Screenshot of Test Cases

► Proposed grade: 10 / 10

▼ Comments

Summary of tests

```
+-----+
| 3 tests run/ 3 tests passed |
+-----+
```

SCOPE / BCSE202P_VL2025260103899 / Recursion Assignment / 1

VPL 1

Description

</> Edit

Submission view

Grade

Reviewed on Saturday, 6 September 2025, 12:21 AM by Automatic grade
Grade: 10.00 / 10.00

Assessment report [-]

[+] Summary of tests

Submitted on Saturday, 6 September 2025, 12:21 AM (Download)

1.c

```
1 #include<stdio.h>
2 #define max 100
3
4 int m,n;
5 int board[max][max];
6 int visited[max][max];
7
8 int dp[max][max][max][max][2];
9 int calculated[max][max][max][max][2];
10
11 int dx[4] = {-1, 1, 0, 0};
12 int dy[4] = {0, 0, -1, 1};
13
14 int solve(int x1, int y1, int x2, int y2, int turn, int score1);
15
16 int myMax(int a, int b){
17     if(a > b){
18         return a;
19     }
20     else{
21         return b;
22     }
23 }
24
25 int myMin(int a, int b){
26     if(a < b){
27         return a;
28     }
29     else{
30         return b;
31     }
32 }
33
34 void readMatrix(int i, int j){
35     if(i == m){
36         return;
37     }
38     if(j == n){
39         readMatrix(i + 1, 0);
40         return;
41     }
42     scanf("%d", &board[i][j]);
```

Question 3

Problem Statement

You are given an M

timesN matrix filled with characters 'X' and 'O'. Your task is to capture all regions of 'O's that are "surrounded".

A region is captured by flipping all 'O's in that region to 'X's. A region of 'O's is considered surrounded if it is not connected to any 'O' on the border of the grid. A connection can be to an adjacent cell (up, down, left, or right).

The operation should be performed in-place on the given matrix.

Input Format

An MtimesN matrix of characters ('X' and 'O').

Output Format

The same matrix, with the surrounded 'O' regions flipped to 'X'.

Sample Case

Input Matrix:

```
[['X', 'X', 'X', 'X', 'X'],
 ['X', 'O', 'O', 'X', 'O'],
 ['X', 'X', 'O', 'X', 'O'],
 ['X', 'O', 'X', 'O', 'X'],
 ['O', 'O', 'O', 'X', 'X']]
```

Expected Output (in-place modification):

```
[['X', 'X', 'X', 'X', 'X'],
 ['X', 'O', 'O', 'X', 'O'],
 ['X', 'X', 'O', 'X', 'O'],
 ['X', 'O', 'X', 'X', 'X'],
 ['O', 'O', 'O', 'X', 'X']]
```

Explanation:

The 'O's on the border (e.g., at (4,0) and (1,4)) are "safe". Any region of 'O's connected to these safe 'O's is also safe.

The large cluster of 'O's in the bottom-left is connected to the 'O' at (4,0) and is therefore safe. The two 'O's on the right edge are on the border, so they are safe. The only 'O' that is not connected to any border 'O's is the one at (3,3). It forms a surrounded region of size 1 and is flipped to 'X'.

Code

```
#include<stdio.h>
#define max 20

int m,n;
char board[max][max];
int visited[max][max];

void readMatrix(int i, int j){
    if(i == m){
        return;
    }
    if(j == n){
        readMatrix(i+1,0);
        return;
    }

    scanf(" %c",&board[i][j]);
    visited[i][j] = 0;
    readMatrix(i,j+1);
}

int checkRegion(int x, int y, int *cellX, int *cellY, int *count){
    if(x < 0 || x >= m || y < 0 || y >= n){
        return 0;
    }
    if(board[x][y] != 'O' || visited[x][y]){
        return 0;
    }

    visited[x][y] = 1;
    cellX[*count] = x;
    cellY[*count] = y;
    (*count)++;

    int border = (x == 0 || x == m-1 || y==0 || y== n-1);

    border |= checkRegion(x-1,y,cellX,cellY,count);
    border |= checkRegion(x+1,y,cellX,cellY,count);
    border |= checkRegion(x,y-1,cellX,cellY,count);
    border |= checkRegion(x,y+1,cellX,cellY,count);

    return border;
}

void solve(){
    for(int i = 0; i < m; i++){
        for(int j = 0; j < n; j++){
            if(board[i][j] == 'O' && !visited[i][j]){
                int cellX[max*max], cellY[max*max],count = 0;
                int touchBorder = checkRegion(i,j,cellX, cellY, &count);

                if(!touchBorder){
                    for(int k = 0;k < count; k++){
                        board[cellX[k]][cellY[k]] = 'X';
                    }
                }
            }
        }
    }
}
```

```
    }  
  }  
}  
}
```

```
void printMatrix(int i,int j){  
  if( i == m){  
    return;  
  }  
  if(j == n){  
    printf("\n");  
    printMatrix(i+1,0);  
    return;  
  }  
  printf(" %c",board[i][j]);  
  printMatrix(i,j+1);  
}
```

```
int main(){  
  scanf("%d %d",&m,&n);  
  
  readMatrix(0,0);  
  solve();  
  printMatrix(0,0);  
  
  return 0;  
}
```

Code Screenshot

```
1 #include<stdio.h>
2 #define max 20
3
4 int m,n;
5 char board[max][max];
6 int visited[max][max];
7
8 void readMatrix(int i, int j){
9     if(i == m){
10         return;
11     }
12     if(j == n){
13         readMatrix(i+1,0);
14         return;
15     }
16     scanf(" %c",&board[i][j]);
17     visited[i][j] = 0;
18     readMatrix(i,j+1);
19 }
20
21 int checkRegion(int x, int y, int *cellX, int *cellY, int *count){
22     if(x < 0 || x >= m || y < 0 || y >= n){
23         return 0;
24     }
25     if(board[x][y] != 'O' || visited[x][y]){
26         return 0;
27     }
28     visited[x][y] = 1;
29     cellX[*count] = x;
30     cellY[*count] = y;
31     (*count)++;
32
33     int border = (x == 0 || x == m-1 || y==0 || y== n-1);
34
35     border |= checkRegion(x-1,y,cellX,cellY,count);
36     border |= checkRegion(x+1,y,cellX,cellY,count);
37     border |= checkRegion(x,y-1,cellX,cellY,count);
38     border |= checkRegion(x,y+1,cellX,cellY,count);
39
40     return border;
41 }
42
43 void solve(){
44     for(int i = 0; i < m; i++){
45         for(int j = 0; j < n; j++){
46             if(board[i][j] == 'O' && !visited[i][j]){
47                 int cellX[max*max], cellY[max*max], count = 0;
48                 int touchBorder = checkRegion(i,j,cellX, cellY, &count);
49                 if(touchBorder){
50                     for(int k = 0; k < count; k++){
51                         board[cellX[k]][cellY[k]] = 'X';
52                     }
53                 }
54             }
55         }
56     }
57 }
58
59 void printMatrix(int i,int j){
60     if(i == m){
61         return;
62     }
63     if(j == n){
64         printf("\n");
65         printMatrix(i+1,0);
66         return;
67     }
68     printf("%c",board[i][j]);
69     printMatrix(i,j+1);
70 }
71
72 int main(){
73     scanf("%d %d",&m,&n);
74     readMatrix(0,0);
75     solve();
76     printMatrix(0,0);
77
78     return 0;
79 }
```

Screenshot of Test Cases

► Proposed grade: 10 / 10

▼ Comments

Summary of tests
+-----+
| 5 tests run/ 5 tests passed |
+-----+

Grade

Reviewed on Sunday, 7 September 2025, 12:11 AM by Automatic grade

Grade: 10.00 / 10.00

Assessment report  [-]

[+]Summary of tests

 Submitted on Sunday, 7 September 2025, 12:11 AM ( Download)

3.c

```
1  #include<stdio.h>
2  #define max 20
3
4  int m,n;
5  char board[max][max];
6  int visited[max][max];
7
8  void readMatrix(int i, int j){
9      if(i == m){
10         return;
11     }
12     if(j == n){
13         readMatrix(i+1,0);
14         return;
15     }
16
17     scanf(" %c",&board[i][j]);
18     visited[i][j] = 0;
19     readMatrix(i,j+1);
20 }
21
22 int checkRegion(int x, int y, int *cellX, int *cellY, int *count){
23     if(x < 0 || x >= m || y < 0 || y >= n){
24         return 0;
25     }
26     if(board[x][y] != '0' || visited[x][y]){
27         return 0;
28     }
29
30     visited[x][y] = 1;
31     cellX[*count] = x;
32     cellY[*count] = y;
33     (*count)++;
34
35     int border = (x == 0 || x == m-1 || y==0 || y== n-1);
36
37     border |= checkRegion(x-1,y,cellX,cellY,count);
38     border |= checkRegion(x+1,y,cellX,cellY,count);
```